

Design and Synthesis of 2:1 Mux using Yosys

INTRODUCTION:

This lab provides an introduction to synthesizing a 2:1 Multiplexer using Yosys. We will learn how to read the design, perform optimization, map it to a standard cell library, and generate the gate-level netlist.

The exercise helps in understanding how RTL code is transformed into hardware-efficient logic.

APPLICATION:

2:1 multiplexers are fundamental building blocks in digital systems, used for data selection, signal routing, and control. Their simplicity, speed, and flexibility make them essential in combinational and sequential circuits, as well as larger designs like multiplexers, demultiplexers, and PLDs.

OBJECTIVE:

After completing this lab, Student will be able to:

- Write Verilog code for a 2:1 Multiplexer.
- Load and synthesize the design using Yosys.
- Generate and analyze the gate-level netlist.
- Understand mapping of the design to standard cells.
- Verify correctness of synthesis through reports and outputs

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

- 1.1: Create a directory : `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir 2by1mux`
- 1.4: Open project directory: `cd 2by1mux`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim mux_2x1.v`

RTL Design Code:

```
module mux_2x1 (  
    input a,  
    input b,  
    input sel,  
    output out);  
    assign out = sel ? b : a;  
endmodule
```

2.2: Save the file using command- :wq!

Step 3: Synthesize the Design with Yosys

invoke Yosys in a new terminal by the following command - yosys

3.1: Use the following command to Synthesize:

```
# Read the Verilog source file  
read_verilog mux_2x1.v  
# Set the top-level module and check hierarchy  
hierarchy -check -top mux_2x1  
# high level stuff  
proc; opt; map; opt;  
# mapping to internal cell library  
techmap; opt  
# mapping flip-flops to mycells.lib  
dfflibmap -liberty osu018_stdcells.lib  
# mapping logic to mycells.lib  
abc -liberty osu018_stdcells.lib  
# Clean up unused cells and signals after mapping  
clean  
# Write the final synthesized gate-level netlist to a new Verilog file
```

```
write_verilog mux_2x1_synth.v
```

```
#Generate a graphical visualization of the synthesized circuit
```

```
show
```

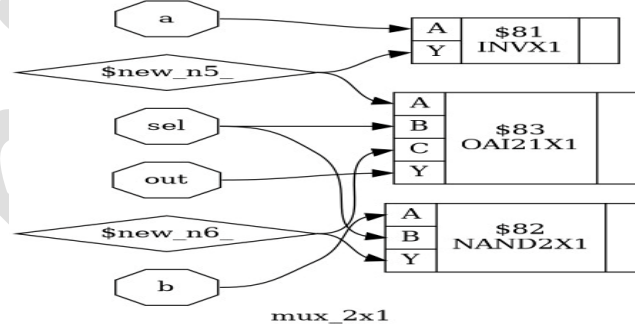
Step 4: View Output File (Netlist)

```
gvim mux_2x1_synth.v
```

OBSERVATION:

- The design is synthesized without errors.
- The multiplexer logic is correctly implemented using combinational gates.
- The select line properly controls the output to choose between the two inputs.
- The synthesized netlist confirms correct mapping of the MUX functionality into the target library.

OUTPUT:



CONCLUSION:

The Yosys synthesis output confirms the correct implementation of the 2:1 multiplexer. The RTL design is successfully read, optimized, and mapped to the target standard cell library. The generated gate-level netlist shows that the multiplexer logic is preserved, with the output correctly determined by the select line to choose between the two inputs. The synthesis report verifies that the selection functionality is properly integrated, ensuring accurate data routing.